Manuale del Progettista – Progetto Ruggine

Introduzione

Ruggine è un sistema di chat di gruppo **client-server** scritto in Rust, pensato per la comunicazione testuale in tempo reale tra più utenti. Si compone di un **server** centrale e di **client** multipli che possono connettersi per scambiarsi messaggi. Il server gestisce la creazione di gruppi di chat, l'invio di **inviti** ad altri utenti e il instradamento dei messaggi verso i membri appropriati. I client sono applicazioni desktop: principalmente un'app con **interfaccia grafica (GUI)** realizzata con il framework Iced, tramite la quale gli utenti possono registrarsi (login), creare gruppi, invitare partecipanti e inviare messaggi. La comunicazione tra client e server avviene su **connessioni TCP** usando un semplice protocollo testuale: ogni messaggio applicativo è serializzato in **JSON** e terminato da un carattere newline `\n`.

Il progetto adotta tecnologie Rust moderne per ottenere alta **concorrenza** e reattività: il runtime **Tokio** viene utilizzato per la gestione asincrona delle connessioni di rete e dei task concorrenti, il framework **Iced** per la GUI reattiva del client, la libreria **serde_json** per serializzare e deserializzare i messaggi JSON, e la crate **sysinfo** per monitorare le risorse di sistema (in particolare l'utilizzo della CPU sul server).

Oltre al client grafico, per facilitare lo sviluppo incrementale e i test è stato realizzato anche un **client testuale** a riga di comando (CLI) che permette di interagire col server senza GUI. Nelle sezioni seguenti verranno descritti in dettaglio l'architettura generale, i principali moduli software, le scelte tecnologiche adottate e i flussi operativi più importanti (login, creazione gruppi, inviti, chat, logging delle prestazioni CPU), fornendo una documentazione tecnica completa e professionale del progetto.

Architettura del Sistema

Ruggine segue il classico paradigma **client-server** distribuito: un unico server centrale accetta le connessioni dai client e coordina tutte le operazioni, mentre i client si occupano dell'interazione con l'utente finale. In particolare, il **server** mantiene lo stato globale dell'applicazione (elenco utenti attivi, gruppi esistenti, inviti pendenti) e si occupa di ricevere i messaggi dai client e inoltrarli ai destinatari appropriati (ad esempio a tutti i membri di un gruppo durante una chat). Dal lato opposto, ciascun **client** fornisce un'interfaccia tramite cui l'utente può effettuare il login con un nome univoco, creare nuovi gruppi di chat, invitare altri utenti nei propri gruppi, unirsi a gruppi a cui è stato invitato e inviare messaggi di testo in tempo reale.

La comunicazione di rete avviene mediante **socket TCP** su una porta nota (8080 di default) e utilizza un protocollo applicativo basato su **messaggi JSON delimitati da newline**. Ogni messaggio significativo (login, comando di gruppo, chat, ecc.) è rappresentato da un elemento di un **enum** condiviso (definito nel modulo `common`) chiamato `Message`, serializzato in JSON tramite Serde e terminato da `\n` prima di essere inviato. Il newline funge da separatore tra messaggi consecutivi sul flusso TCP, in modo che il destinatario possa usare un `BufReader` per leggere linee intere e ricostruire ciascun JSON. Questo formato testuale rende il protocollo **leggibile e estensibile**, facilitando debug e possibili aggiunte future di nuovi campi o tipologie di messaggio.

Dal punto di vista della **concorrenza**, il server è progettato per gestire molteplici client in parallelo sfruttando la **programmazione asincrona** di Tokio. Ogni nuova connessione viene servita dal server avviando un task asincrono dedicato (tramite `tokio::spawn`), senza bloccare il thread principale. Internamente, per condividere lo stato globale tra questi task (ad esempio la lista degli utenti online, le membership dei gruppi e gli inviti), il server utilizza strutture dati protette da **arcoggetti atomici con mutua esclusione** (`Arc<Mutex<...>>`): in questo modo più task concorrenti possono accedere in sicurezza alle stesse mappe e liste. Questo approccio introduce un piccolo overhead di sincronizzazione ma garantisce la **correttezza** evitando condizioni di gara (race conditions) sui dati condivisi. I client, dal canto loro, beneficiano anch'essi di Tokio (ad esempio il client testuale è interamente asincrono, e il client GUI integra Tokio per la comunicazione di rete), in modo da restare reattivi alle azioni utente senza bloccare l'interfaccia.

Un ulteriore componente architetturale è il **logging periodico delle prestazioni** sul server. È stato implementato un task asincrono separato che ad intervalli regolari

misura l'utilizzo globale di **CPU** e lo registra in un file di log. Questa funzionalità di monitoraggio sfrutta la crate `sysinfo` per leggere i dati di carico CPU e mostra come eseguire compiti periodici nell'ambiente Tokio (timer e spawn di task ricorrenti). Il file generato (`cpu.log`) consente di analizzare nel tempo le prestazioni del server in termini di consumo di CPU, ed è utile per capire l'andamento sotto carico.

Componenti Principali e Scelte Tecnologiche

Di seguito sono elencati i moduli e componenti principali che costituiscono il sistema Ruggine, con il loro ruolo e le tecnologie chiave utilizzate:

Modulo	Ruolo/Funzionalità	Tecnologie Utilizzate
Server	Applicazione server asincrona che accetta connessioni TCP dai client, gestisce gli **utenti** registrati, i **gruppi** di chat, gli **inviti** pendenti e inoltra i messaggi di chat ai destinatari corretti. Include anche un task periodico per il monitoraggio della CPU.	Tokio (runtime asincrono), serde_json (serializzazione JSON), sysinfo (logging CPU), Arc<Mutex> (condivisione dati concorrente)
Client GUI	Applicazione client **grafica** desktop per l'utente finale. Fornisce un'interfaccia interattiva (finestre, pulsanti, liste) per effettuare il login, visualizzare i gruppi disponibili, chattare e gestire inviti, inviando i relativi comandi al server tramite rete.	Iced (framework GUI reattivo), Tokio (per integrazione I/O asincrono), serde_json (messaggistica JSON)
Client testuale (CLI)	Applicazione client a **riga di comando** usata per test e debugging. Consente di connettersi al server e inviare manualmente comandi testuali (`/login`, `/create`, `/join`, ecc.), visualizzando in console i messaggi ricevuti.	Tokio (I/O asincrono su STDIN/STDOUT e socket), serde_json (messaggistica JSON)

Modulo	Ruolo/Funzionalità	Tecnologie Utilizzate
Libreria Common	Modulo condiviso che definisce il **protocollo di comunicazione** tra client e server. Contiene l'enumerazione `Message` con tutte le varianti di messaggio (login, join, text, ecc.), serializzabile in JSON su entrambi i lati.	Serde (derive Serialize/Deserialize per `Message`), serde_json (serializzazione/deserializzazione)
Modulo Logging CPU	Componente del server per il **monitoraggio delle risorse**. Esegue periodicamente la raccolta dell'utilizzo CPU e lo scrive su file di log, così da fornire dati sulle prestazioni del server nel tempo.	Tokio (timer asincrono con task periodico), sysinfo (lettura metriche di sistema)

Come evidenziato, la scelta tecnologica chiave è l'uso di **Rust async** con Tokio per ottenere un sistema non bloccante e altamente **scalabile**: in teoria il server può servire decine o centinaia di client concorrenti, essendo ogni connessione gestita da un task separato e le attese di I/O sono tutte asincrone. L'**interfaccia grafica** è realizzata con Iced, un moderno framework in stile Elm/Redux che garantisce separazione tra logica e presentazione e facilita la gestione dello stato dell'applicazione GUI tramite messaggi ed eventi. Per il **formato dati**, l'uso di JSON (via Serde) offre un protocollo semplice da leggere e diagnosticare, oltre che facile da estendere aggiungendo nuovi campi o tipi di messaggio senza rompere la compatibilità. Infine, la crate **sysinfo** fornisce un accesso portabile a informazioni di sistema (CPU, memoria, ecc.) ed è utilizzata per implementare il logging delle prestazioni CPU sul server.

Protocollo di Comunicazione e Formato dei Messaggi

L'interazione tra client e server avviene tramite lo scambio di messaggi testuali JSON su TCP. Il **protocollo** è definito dall'enum `common::Message`, condiviso sia dal server che dai client, che elenca tutte le possibili operazioni/comandi e notifiche del sistema. Ciascun messaggio viene convertito in una stringa JSON e inviato seguito da un newline `\n`; il newline funge da delimitatore così che il ricevente possa individuare la fine di ogni messaggio sul socket. Di seguito sono riportati i

principali tipi di **Message** e il loro significato (i campi indicati tra parentesi tonde sono quelli contenuti nel JSON):

- **Login** `(username)` – Inviato da un client quando effettua il login al sistema specificando il proprio nome utente. Il server registra la connessione associandola a tale username e risponde inviando un messaggio di benvenuto o di aiuto (oltre a eventuali inviti pendenti per quell'utente). Questo è generalmente il primo messaggio scambiato da un client appena connesso.
- **Create** `(group)` – Inviato da un client per creare un **nuovo gruppo** di chat con nome `group`. Il server crea la nuova entry di gruppo nel suo stato e aggiunge automaticamente il creatore come membro (impostandolo anche come **invitato** di quel gruppo). Al client viene restituito un *acknowledgment* (Message::Ack) per confermare che la creazione è avvenuta con successo. Dopo questo comando, il creatore potrà unirsi al gruppo (opzionalmente il client GUI lo fa in automatico).
- **Join** `(username, group)` – Richiesta di un utente di **unirsi** a un gruppo esistente. Viene inviata da un client (con il proprio username e il nome del gruppo) tipicamente in risposta a un invito ricevuto. Il server verifica se l'utente risulta invitato a quel gruppo (controllando se il suo username è presente nella lista degli inviti pendenti per il gruppo). Se l'invito non c'è, il server risponde con un *errore* (Message::Error) indicando che l'accesso non è autorizzato; se invece l'invito è presente, l'utente viene aggiunto come membro attivo del gruppo (inserendo la sua connessione nella lista dei writer del gruppo) e il server invia un Ack di conferma. Dopo un Join riuscito, il nuovo membro inizierà a ricevere tutti i messaggi destinati al gruppo.
- **Leave** `(group)` – Inviato da un client per **lasciare** un gruppo di cui fa parte. Il server, alla ricezione, rimuove l'utente (la sua connessione) dalla lista dei membri del gruppo indicato. Inoltre inoltra a tutti gli altri membri del gruppo un messaggio di sistema (Message::Text con mittente "system") per notificare che l'utente ha abbandonato la chat (es. *"Alice ha lasciato il gruppo"*). Infine il server risponde all'utente che ha inviato il Leave con un Ack di conferma. Dopo ciò, l'utente non riceverà più messaggi da quel gruppo a meno che non vi si unisca nuovamente.
- **Text** `(group, from, content)` – È il messaggio di **chat** vero e proprio, contenente un testo utente. Un client lo invia per pubblicare il contenuto `content` all'interno del

gruppo `group`. Il server, ricevuto il Message::Text, verifica che il mittente `from` sia effettivamente membro attivo di quel gruppo. Se non lo è (ad esempio per un errore logico o un tentativo malevolo), risponde con un Message::Error al mittente (*"non sei autorizzato a inviare in questo gruppo"*) senza inoltrare nulla. Se invece l'utente è membro, il server inoltra il messaggio a **tutti i membri connessi** del gruppo, serializzandolo di nuovo come JSON e inviandolo sulle rispettive socket. Ogni client membro quindi riceverà una copia del messaggio (incluso il mittente stesso, che lo riceve indietro dal server). Il server non mantiene uno **storico** dei messaggi: la trasmissione avviene in tempo reale ai client online; un client offline non recupererà i messaggi persi durante la sua assenza.

- **Invite** `(group, user)` – Inviato da un client per **invitare** un altro utente ad unirsi a un gruppo. Tipicamente questo comando è disponibile solo a chi è già membro del gruppo. Il server, ricevuto l'invito, determina l'username dell'**invitante** (chi ha mandato il comando) e verifica che egli faccia parte del gruppo indicato. Se l'invitante *non* è membro, il server risponde con un Message::Error (es. *"Autorizzazione negata: non sei membro del gruppo X"*) e ignora la richiesta. Se invece l'invitante è un membro autorizzato, il server registra l'invito aggiungendo il nome `user` nella struttura degli inviti pendenti per quel gruppo (`invites[group]`). Inoltre, controlla se l'utente invitato è attualmente online (presente nella mappa degli utenti connessi); in tal caso gli invia immediatamente un messaggio Message::Invite con il nome del gruppo, in modo che il suo client possa notificargli in tempo reale l'invito ricevuto. Se l'utente non è online, l'invito rimarrà memorizzato e gli verrà inviato non appena si collegherà (durante la procedura di login, il server inoltra tutti gli inviti pendenti come visto sopra per il comando Login).
- **Ack** – È un *acknowledgment* generico inviato **dal server al client** per confermare che un'operazione richiesta è stata eseguita con successo. Il server usa Message::Ack come risposta positiva per vari comandi: ad esempio dopo un Create group, un Join o un Leave, il client riceve un Ack a indicare che il server ha completato l'azione. Il messaggio Ack non contiene campi aggiuntivi, ma il client può dedurre l'operazione confermata tenendo traccia della richiesta fatta (es. se l'utente aveva appena mandato Join, l'arrivo di un Ack implica che l'ingresso nel gruppo è avvenuto).

- **Error** `(reason)` – È il corrispettivo negativo dell'Ack, inviato **dal server al client** quando una richiesta non può essere soddisfatta a causa di un errore o di una violazione delle regole. Il campo `reason` contiene una stringa descrittiva dell'errore (ad esempio: *"Impossibile unirsi: non sei stato invitato al gruppo X"* in caso di Join senza invito). Il server invia Message::Error solo al client che ha generato l'azione non valida, così che l'interfaccia utente di quel client possa mostrare un avviso o messaggio d'errore all'utente finale. I client normalmente non inviano messaggi Error tra loro (sono solo risposte server-side).

Nel design di Ruggine tutti i messaggi viaggiano in formato testo JSON. L'**enumerazione** `Message` definita nel modulo comune funge da **schema**: grazie a Serde, ogni variant viene convertita in JSON con campi nominati, e al contrario il JSON ricevuto viene convertito (deserializzato) di nuovo in un `Message` tipizzato. Ciò rende l'implementazione del protocollo semplice da gestire in codice (basta fare matching sul tipo di `Message` ricevuto). La scelta di un protocollo testuale (invece che binario) ha anche facilitato il debugging, permettendo di vedere chiaramente in console i JSON scambiati tra le parti. La terminazione a newline assicura che non ci siano ambiguità nei confini dei messaggi, pur mantenendo la possibilità di includere spazi e altri caratteri nel content dei messaggi di chat.

Server: Struttura e Funzionamento

Il **server Ruggine** è implementato come un eseguibile Rust asincrono (crate `server`) che fa uso intensivo di Tokio. La funzione `main` del server esegue le inizializzazioni necessarie e poi entra in un loop di servizio delle connessioni. In pseudocodice, il funzionamento generale del server è il seguente:

1. **Avvio del listener TCP** – Viene aperto un `TcpListener` in ascolto sull'indirizzo `0.0.0.0:8080` (tutte le interfacce, porta 8080). Se il binding ha successo, il server sarà pronto ad accettare nuove connessioni in ingresso.

2. **Avvio del task di logging CPU** – In parallelo, subito dopo, il server lancia (con `tokio::spawn`) un task asincrono dedicato al **logging periodico** delle prestazioni. Questo task utilizza un timer (`tokio::time::interval`) per svegliarsi ogni 120 secondi e invocare la funzione `log_cpu_usage()`. Tale funzione (definita nel modulo `logging::cpu`) raccoglie l'utilizzo corrente di CPU e lo appende nel file `cpu.log`. In caso di errore di I/O nel log, viene stampato un messaggio di

errore su stderr, ma il server continua l'esecuzione. Questo meccanismo fornisce un monitoraggio continuo del carico del server senza interferire con la gestione delle connessioni.

3. **Inizializzazione delle strutture dati condivise** – Il server prepara le strutture dati globali che terranno traccia dello stato: in particolare vengono create tre mappe, tutte incapsulate in un `Arc<Mutex<...>>` per permetterne l'accesso concorrente dai vari task. Nel dettaglio:

   - `users: HashMap<String, Arc<Mutex<OwnedWriteHalf>>>` – mappa ogni **username** di client connesso a un writer (la metà scrittura della sua socket TCP) incapsulato in un Mutex protetto e condiviso. In questo modo il server può, dato il nome di un utente, ottenere un riferimento sicuro al canale di output verso quel client, su cui inviare eventuali messaggi.

   - `groups: HashMap<String, Vec<Arc<Mutex<OwnedWriteHalf>>>>>` – mappa ogni **nome di gruppo** a un vettore di writer (connessioni) dei membri attivi di quel gruppo. Ogni qual volta un utente si unisce a un gruppo, il suo writer viene aggiunto alla lista del gruppo; quando lascia, viene rimosso. Questo permette al server di inoltrare i messaggi di chat ad ogni membro del gruppo iterando sul vettore associato.

   - `invites: HashMap<String, HashSet<String>>` – mappa ogni **nome di gruppo** all'insieme degli **username invitati** (ma non ancora uniti) a quel gruppo. Quando un utente invita qualcuno in un gruppo, viene aggiunto qui; se poi l'invitato si collega e fa join, il suo nome potrà essere rimosso da questo set. Questi inviti "pendenti" servono a autorizzare le richieste di ingresso nei gruppi e a notificare gli inviti non appena l'utente invitato si connette.

   L'uso di `Arc<Mutex<...>>` per ognuna di queste mappe consente di **condividere** tali strutture tra i vari task Tokio in esecuzione (ogni task che gestisce un client ne deterrà una copia dell'Arc). Il Mutex garantisce mutua esclusione per evitare corruzione dei dati qualora più eventi concorrenti (es. due utenti che creano gruppi contemporaneamente) dovessero aggiornare le strutture. In casi semplici come questo progetto, un Mutex su ciascuna mappa offre sufficiente sicurezza, anche se in scenari altamente concorrenti potrebbe rappresentare un collo di bottiglia (ad esempio molti utenti che accedono in scrittura allo stesso gruppo contemporaneamente potrebbero

accodarsi sul lock). Per scopi di chiarezza e robustezza iniziale, però, questa soluzione garantisce correttezza e facilita la comprensione del flusso dati.

4. **Loop di accettazione connessioni** – Il server entra in un loop infinito dove chiama `listener.accept().await` per accettare ogni nuova connessione client in arrivo. Quando una connessione viene accettata, il server estrae un `TcpStream` e ne ricava separatamente l'oggetto di lettura e quello di scrittura tramite `stream.into_split()`. Ottiene così un `OwnedReadHalf` e un `OwnedWriteHalf` indipendenti. Il writer viene incapsulato in un `Arc<Mutex<OwnedWriteHalf>>` (in modo simile ai container delle mappe globali) così che possa essere condiviso facilmente. A questo punto il server crea delle copie (Arc clone) dei riferimenti alle tre strutture globali (`users`, `groups`, `invites`) e lancia un nuovo task Tokio asincrono dedicato a quella connessione, invocando la funzione `handle_client(reader, writer, addr, groups, users, invites)` in un task separato. Il parametro `addr` è l'indirizzo socket del client appena connesso (usato solo per log). In caso di errore durante la gestione del client (eccezione non recuperata all'interno di handle_client), viene stampato un messaggio di errore con l'indirizzo del client interessato, ma il loop principale continua ad accettare nuove connessioni. Grazie a questo schema, il server può **servire molteplici client contemporaneamente**: ogni connessione è gestita dal proprio task indipendente, e le attese di I/O (lettura socket, ecc.) non bloccano l'esecuzione delle altre.

Gestione delle Richieste dei Client

La logica principale del server risiede nella funzione asincrona `handle_client(...)`, che si occupa di gestire il ciclo di vita di una singola connessione con un client. In termini generali, `handle_client` esegue un **loop di lettura** sul socket del client e reagisce ai messaggi ricevuti in base al loro contenuto. Di seguito descriviamo il comportamento passo-passo di questo ciclo, quindi i casi specifici per ciascun tipo di messaggio:

- **Lettura del messaggio** – All'interno di `handle_client`, il server crea un `BufReader` attorno all'`OwnedReadHalf` del socket, così da poter comodamente leggere linee di testo dal flusso. Entra quindi in un ciclo `loop` dove, ad ogni iterazione, chiama `read_line(&mut line).await` sul BufReader per leggere una linea completa dal client. Se `read_line` restituisce 0 byte, significa che il client ha chiuso la connessione (EOF raggiunto); in tal caso il server esce dal loop,

stampa un messaggio di disconnessione per log (es. *"Client 127.0.0.1:12345 disconnesso"*) e termina il task di handle_client. Se invece viene letta correttamente una linea, il contenuto (stringa JSON) viene memorizzato in `line`. Il server applica un `.trim_end()` per rimuovere il newline terminale e ottiene così la stringa JSON pura. Può essere utile a fini di debug stampare a console il JSON ricevuto; nel codice è presente ad esempio un log con prefisso "🚀 Ricevuto raw JSON: ..." per ogni messaggio letto.

- **Parsing JSON in Message** – Una volta ottenuta la stringa JSON, il server tenta di **deserializzarla** nel tipo `Message` usando `serde_json::from_str`. Questo potrebbe fallire se il client invia dati malformati; il codice prevede tale eventualità: in caso di errore di parsing, viene stampato su stderr un messaggio di errore che include l'indirizzo del client e la descrizione dell'eccezione JSON. Dopodiché il loop continua alla prossima iterazione (il messaggio malformato viene scartato). Se invece il parsing ha successo, si ottiene una variante dell'enum `Message` che rappresenta il comando/richiesta effettivamente inviata dal client.
- **Gestione in base al tipo di messaggio** – Tramite un `match` sul valore di tipo `Message` deserializzato, il server distingue diversi casi e applica la logica di conseguenza. Analizziamo singolarmente le azioni intraprese per ciascun tipo di messaggio (corrispondenti ai comandi descritti prima dal punto di vista del protocollo):
  - **Login:** quando il server riceve `Message::Login { username }`, significa che un nuovo client sta effettuando l'accesso con l'identificativo indicato. La procedura eseguita è:
    1. Acquisire il lock su `users` e inserire una nuova entry con chiave = `username` e valore = `writer` (il riferimento condiviso alla metà scrittura di quella connessione). Ciò registra l'utente come **connesso**. Se lo username era già presente, l'associazione viene aggiornata (ma in questo scenario è presumibile che due utenti non scelgano lo stesso nome, non essendoci un controllo esplicito il comportamento in caso di duplicati potrebbe essere che l'ultimo writer sovrascrive il precedente).
    2. Inviare sul socket del client un **messaggio di testo di benvenuto/aiuto** con la lista dei comandi disponibili. Nel codice questo è realizzato creando un `Message::Text` con `group = "system"` (per indicare

che è un messaggio di sistema) e `from = "server"`, contenente nel campo `content` una stringa multilinea con i comandi come `/create <gruppo>`, `/invite <user> <gruppo>`, `/join <gruppo>`, ecc.. Questo messaggio viene serializzato in JSON e spedito immediatamente al client (seguito da `\n`). In tal modo, all'atto del login l'utente ottiene subito istruzioni su cosa può fare nel sistema.

3. Controllare se esistono **inviti pendenti** per l'utente che sta facendo login. Il server prende un lock su `invites` e scorre tutti i gruppi nella mappa cercando il nome utente in ciascun set. Per ogni gruppo in cui l'utente compare come invitato, il server invia al client un messaggio `Message::Invite { group, user }` (dove `user` sarà proprio l'username del destinatario) per notificargli che è stato invitato a quel gruppo. In pratica il client, subito dopo il login, riceverà tanti messaggi di invito quanti erano gli inviti in sospeso, potendo così vedere nell'interfaccia i gruppi a cui può unirsi. Dopo aver inviato gli inviti pendenti, l'operazione di login sul server è completa (non viene inviato alcun Ack esplicito per il login, dato che l'invio del messaggio di sistema funge già da segnale di avvenuta accettazione).

- **Create Group:** per un messaggio `Message::Create { group }`, inviato da un client che vuole creare un nuovo gruppo chiamato `group`, il server procede così:

  1. Determina quale utente (username) sta effettuando la richiesta. Poiché il messaggio Create non riporta l'username (il client invia solo il nome del gruppo da creare), il server deve dedurlo. Usa quindi la mappa `users` cercando quale entry contiene come valore quel particolare writer (confronto tramite `Arc::ptr_eq` sugli Arc di writer). Se trova corrispondenza, ottiene il nome utente del creatore; in caso improbabile che non lo trovi (il writer non corrisponde a nessun utente connesso, ad esempio se il login non era stato registrato), il creatore resta sconosciuto. Nella normale operatività, invece, il creatore viene individuato correttamente.

  2. Aggiorna i dati di stato: innanzitutto registra l'informazione dell'invito automatico creando (se non esiste) un entry in `invites` per quel gruppo e inserendovi il nome del creatore. Questo perché, concettualmente, il creatore di un gruppo è immediatamente considerato invitato al gruppo

stesso (in modo che possa fare join senza problemi). In secondo luogo, aggiorna la mappa `groups`: crea una nuova voce `groups[group]` con una lista vuota se il gruppo non esisteva, quindi vi aggiunge il writer del creatore come primo membro del gruppo. In questo modo, si considera il creatore **già membro** del gruppo appena creato. (Nota: nel codice, l'utente creatore viene aggiunto sia a invites sia a groups, quindi risulterà immediatamente membro; il client comunque eseguirà un Join subito dopo per sincronizzare il suo stato locale).

3. Opzionalmente, loggare su console l'evento di creazione per fini di debug (es. stampa "[Server][Create] group='X' created by 'Alice'"). Infine, inviare un messaggio di conferma al client: viene serializzato e spedito un `Message::Ack`. Questo fa sapere al client che il server ha creato il gruppo con successo. Non viene notificato nessun altro utente in questa fase (essendo un nuovo gruppo, non ci sono altri membri a cui notificare).

Con questa procedura, chi crea un gruppo diventa subito membro attivo del gruppo stesso (dal punto di vista del server). Questo garantisce che possa immediatamente invitare altri o inviare messaggi nel nuovo gruppo. Nel caso in cui il writer del creatore fosse già presente (ad esempio se per un errore la procedura venisse invocata due volte rapidamente), il codice evita duplicazioni controllando che l'elemento non sia già nella lista prima di aggiungerlo.

- **Join Group:** quando arriva un `Message::Join { username, group }` (di solito inviato da un client in risposta a un invito precedentemente ricevuto), il server esegue:

  1. Verifica se l'utente indicato (`username`) **è invitato** al gruppo specificato. Si prende un lock su `invites` e si controlla se nella mappa esiste la chiave `group` e se il valore (HashSet) contiene lo `username`. In base a questo:

     - Se **non** c'è l'invito, significa che l'utente sta cercando di unirsi a un gruppo senza permesso. Il server allora invia immediatamente un `Message::Error` al client, spiegando che non è stato invitato e quindi non può entrare in quel gruppo. L'operazione termina qui con un fallimento segnalato.

    - Se invece l'invito esiste, l'utente è autorizzato a entrare. Allora il server procede:

  2. Aggiungere il membro al gruppo: acquisisce il lock su `groups`, recupera (o crea se non esiste ancora) il vettore dei membri per quel gruppo, e vi aggiunge il writer del nuovo utente. In questo modo quell'utente inizia ufficialmente a far parte del gruppo. Inoltre – implementazione specifica – il server aggiorna anche la mappa `users` reinserendo l'associazione username → writer (questo può sembrare ridondante, dato che l'utente dovrebbe già esistere in `users` dal login, ma serve a garantire che l'utente sia registrato attivo; nel codice attuale questa operazione di reinserimento potrebbe non essere strettamente necessaria).
  3. Inviare un `Ack` di conferma al client. Da quel momento, lato server l'utente è membro a tutti gli effetti (si potrebbe anche rimuovere il suo nome da `invites[group]` poiché l'invito è stato "consumato", ma dal codice completo noto che questa rimozione non è implementata; rimane come possibile miglioramento per coerenza dei dati).

  In sintesi, un messaggio di Join andato a buon fine sposta l'utente dalla condizione di "invitato" a quella di **membro attivo** del gruppo, permettendogli di ricevere i messaggi di chat e le altre notifiche di quel gruppo.

- **Leave Group:** quando un client invia `Message::Leave { group }` per uscire da un gruppo, il server compie i seguenti passi:

  1. Determina quale utente sta lasciando. Poiché il messaggio indica solo il gruppo, il server risale all'username cercando nella mappa `users` quale chiave ha come valore il writer corrente (analogamente a quanto fatto per Create). Se lo trova, ottiene l'username; se per qualche ragione non lo trova, usa un placeholder generico (come "qualcuno").
  2. Rimuove l'utente dal gruppo: prende il lock su `groups` e accede al vettore dei membri del gruppo indicato. Elimina da questo vettore l'elemento corrispondente al writer dell'utente uscente. In tal modo il server non considererà più quell'utente come parte del gruppo.

3. Invia una **notifica di uscita** agli altri membri: costruisce un Message::Text con `group = <nome_gruppo>`, `from = "system"` e `content = "<username> ha lasciato il gruppo"` e lo invia a tutti i writer rimanenti nel gruppo. Così, ogni client ancora connesso a quel gruppo riceverà un messaggio di sistema che avvisa dell'uscita di quell'utente.
4. Manda un Ack di conferma al client che ha fatto Leave, dopodiché il server considera la richiesta completata.

Da quel momento, l'utente non figura più tra i membri del gruppo (ma eventualmente potrebbe rimanere invitato se l'invito originale non era stato rimosso; questo dettaglio implementativo potrebbe essere ottimizzato). Gli altri membri hanno ricevuto l'informazione dell'abbandono, e il sistema mantiene consistenza tra ciò che ogni membro vede (nessun messaggio successivo verrà inviato al membro uscente, e chi è rimasto sa che quell'utente non partecipa più).

- **Text (messaggio di chat):** quando il server riceve un `Message::Text { group, from, content }`, esso deve inoltrare il messaggio di chat a tutti i partecipanti del gruppo. Tuttavia, prima esegue un controllo di sicurezza:

  1. **Verifica appartenenza:** controlla se l'utente mittente `from` è effettivamente un membro del gruppo `group`. Per farlo confronta la lista `groups[group]` con l'entry corrispondente in `users[from]`. In pratica, prende il writer salvato per `from` e verifica se è presente nel vettore dei writer del gruppo. Se il gruppo non esiste o se la condizione non è soddisfatta, significa che l'utente non dovrebbe inviare messaggi lì.
  2. Se l'utente **non è membro** autorizzato, il server crea un Message::Error con reason tipo "Impossibile inviare: non sei membro del gruppo 'X'" e lo invia solo al mittente. Dopodiché scarta il messaggio (non viene inoltrato a nessuno).
  3. Se invece l'utente **è membro** del gruppo, il server procede a inoltrare il messaggio:
     - Serializza di nuovo il `Message::Text` (così come ricevuto, o eventualmente potrebbe aggiungere una marca temporale ma in questo progetto non c'è timestamp) e aggiunge `\n`.

    - Prende un lock in lettura su `groups` per ottenere la lista corrente dei writer dei membri del gruppo.
    - Invia il messaggio a **ciascun writer** in quella lista, iterando sui membri. Ogni invio viene fatto in modo asincrono scrivendo sul socket del relativo client. Anche il mittente, essendo incluso nella lista, riceverà il messaggio indietro (questo può essere utile per confermare localmente l'invio nella GUI, ad esempio).
  4. Il server non invia alcun Ack per i messaggi di chat (ha senso, poiché l'arrivo del messaggio stesso ai client funge da riscontro; inoltre un Ack su ogni chat appesantirebbe inutilmente il traffico).
- **Invite User:** infine, il caso `Message::Invite { group, user }` gestisce l'invito di un utente in un gruppo:
  1. Identifica chi è l'**invitante** (username) usando la stessa tecnica vista in Create: cercando il writer corrente in `users`. Ottenuto il nome di chi invita, verifica che costui sia membro del gruppo:
  2. **Verifica autorizzazione:** controlla se il writer dell'invitante compare nella lista `groups[group]`. Se l'invitante non fa parte del gruppo, allora non ha il permesso di invitare altri in quel gruppo. In tal caso il server risponde immediatamente con un `Message::Error` al mittente (es. *"Autorizzazione negata: non sei membro del gruppo X"*), e termina l'elaborazione.
  3. Se l'invitante è **membro del gruppo**, allora il server procede a registrare l'invito:
     - Inserisce l' `user` (username dell'invitato) nella mappa `invites` per quel gruppo: `invites.entry(group).or_default().insert(user)`. Se la voce per il gruppo non esisteva, viene creata con un nuovo HashSet contenente l'username; se già esisteva, l'username viene aggiunto al set.
     - Quindi controlla nella mappa `users` se l'utente invitato è attualmente connesso. Se esiste un entry `users[user]` (cioè l'utente è online), il server gli invia subito un messaggio `Message::Invite { group, user }`. Questo messaggio in pratica notifica in tempo reale l'invito: il client

destinatario lo vedrà apparire come notifica di essere stato invitato a `group`.

- Se l'utente invitato non è online (`user` non presente in `users`), il server non invia nulla immediatamente. L'invito comunque rimane registrato in `invites[group]` e, come descritto per il login, quando quell'utente si connetterà più tardi e manderà il suo Login, il server gli recapiterà tutti gli inviti pendenti. In questo modo l'invito **"resta in sospeso"** finché il destinatario non lo viene a sapere e decide se unirsi.

4. Non è previsto un Ack per l'invitante: nel caso di successo, il protocollo attualmente non invia alcun riscontro al client che ha inviato l'invito (potrebbe essere una miglioria inviare un Ack di invito inviato). Se c'è un errore (invito non autorizzato), invece, l'invitante riceve un Error come visto.

- **Ack / Error:** come già spiegato, questi due tipi di messaggi non sono originati dai client ma solo dal server come **risposte**. Il server invia **Ack** per confermare la riuscita delle operazioni più importanti (creazione gruppo, adesione a gruppo, abbandono gruppo, etc.). Il client che riceve l'Ack può ad esempio abilitare nella sua interfaccia certe funzionalità (dopo Ack di Join, segnare che ora è membro, ecc.). Gli **Error**, invece, vengono inviati quando un'operazione fallisce per qualche motivo di logica o permessi, e contengono un messaggio testuale di errore. Entrambi questi tipi di messaggio sono gestiti lato client per dare feedback immediato all'utente (vedremo nella sezione client come vengono usati).

In aggiunta alle reazioni ai comandi, il server produce anche dei **log di debug** sul terminale per monitorare la sua attività. Oltre ai messaggi di disconnessione e ai JSON ricevuti già menzionati, stampa su stdout informazioni ogni volta che, ad esempio, un gruppo viene creato o un utente entra/esce da un gruppo (usando `println!` nel codice). Questi log aiutano il programmatore a seguire il flusso degli eventi durante test e collaudo, verificando che lo stato interno del server (mappe aggiornate, ecc.) evolva correttamente. In un ambiente di produzione, tali log potrebbero essere gestiti con un livello di log appropriato (info/debug) o rimossi.

Task di Logging CPU

Come accennato, il server include un modulo di **logging delle performance** che opera in background. Questo componente è implementato nella funzione `log_cpu_usage()` (nel crate `logging`) e nel relativo task Tokio periodico. Approfondiamo brevemente come funziona:

- La funzione `log_cpu_usage()` utilizza la crate **sysinfo** per ottenere informazioni sul sistema. In particolare, crea un oggetto `System` (`System::new_all()`) e richiama `sys.refresh_cpu()` per aggiornare i dati sulle CPU disponibili. Dopodiché invoca `sys.global_cpu_info().cpu_usage()` che restituisce la percentuale di utilizzo **globale** della CPU in quel momento.
- Ottenuto il valore (un numero in virgola mobile), la funzione apre (o crea, se non esiste) un file chiamato `"cpu.log"` in modalità append. Su questo file viene scritto un **nuovo record** sotto forma di riga di testo, contenente la stringa `"CPU Usage: XX.XX%"` con il valore formattato. Ogni chiamata appende quindi una nuova linea con la lettura corrente di CPU.
- Nel `main` del server, come visto, viene creato un task asincrono che ogni 120 secondi chiama questa funzione. L'implementazione usa `tokio::time::interval(Duration::from_secs(120))` per ottenere uno stream temporizzato, e in un loop infinito `interval.tick().await` per aspettare 120 secondi, poi eseguire `log_cpu_usage()`. Se la funzione restituisce errore (ad esempio per un problema di scrittura su file), il task cattura l'Errore e lo stampa su standard error, ma non termina il loop (continuerà al tick successivo).
- Questo meccanismo viene eseguito **in parallelo** al loop di accettazione e ai tasks di handle_client grazie alla natura concorrente di Tokio. Il logging della CPU è completamente trasparente per i client (non coinvolge il network né la logica di chat) ed è pensato per fornire dati utili al progettista. Ad esempio, osservando il file `cpu.log` durante una prova di carico, si può vedere come varia il consumo di CPU del server al crescere del numero di client o di messaggi inoltrati.

Nel contesto di questo progetto, il logging CPU è a scopo di **monitoraggio** e dimostrativo di come integrare attività periodiche in un server asincrono. I dati raccolti potrebbero essere in futuro utilizzati per analisi di performance o attivare meccanismi di scaling (non implementati qui). La semplicità dell'output (solo

percentuale CPU) è sufficiente per avere un'idea di massima del carico; estensioni possibili potrebbero includere logging di memoria, conteggio di utenti attivi, etc.

Client GUI (Iced)

Il principale client di Ruggine è un'applicazione desktop con **interfaccia grafica** realizzata in Rust tramite il framework **Iced**. Questo framework adotta un pattern **Model-View-Update** ispirato a Elm, che ben si presta a gestire interfacce utente reattive in modo dichiarativo. Nel caso del client Ruggine, la struttura logica è la seguente:

- **Model (Modello/Stato):** il client mantiene tutto lo stato dell'applicazione in una struttura `ChatGUI`. Questa struct contiene i campi necessari a rappresentare sia lo stato dell'utente sia quello dell'interfaccia. In particolare, memorizza il **nome utente** corrente (dopo login), l'elenco dei **gruppi disponibili** (a cui l'utente ha accesso o è stato invitato), l'elenco dei **gruppi uniti effettivamente** (quelli in cui ha eseguito il Join), una collezione di **messaggi di chat** per ogni gruppo, i valori attuali dei vari campi di **input** testuale (es. il contenuto del messaggio da inviare, il nome di un nuovo gruppo in fase di creazione, lo username di un utente da invitare) e una serie di **flag booleani** che indicano se certi elementi UI sono attivi/visibili (ad esempio `creating_group` indica se è aperta la finestra popup per creare un nuovo gruppo, `inviting_user` se è aperto il popup per l'invito di un utente). Inoltre c'è un campo `rx` che contiene un ricevitore asincrono (`UnboundedReceiver<Message>`) verso cui il modulo di rete recapita i messaggi ricevuti dal server. Questo modello centralizzato consente di **conservare lo stato** della UI e di poter ricostruire la schermata in qualsiasi momento a partire dai dati (ad esempio dopo un aggiornamento di stato dovuto a un evento di rete).
- **View (Vista):** la parte di visualizzazione dell'interfaccia è definita dalla funzione `view()` dell'applicazione Iced. In questa funzione si costruiscono i vari **widget** (elementi grafici) che compongono la finestra, posizionandoli e legandoli ai dati del modello. Il client Ruggine presenta un'interfaccia divisa in due pannelli principali:
    - Una **barra laterale sinistra**, che mostra in alto il nome utente loggato e un pulsante "Create Group" per creare un nuovo gruppo. Subito sotto c'è l'elenco dei gruppi disponibili (`self.groups`), ciascuno reso come un pulsante

cliccabile (il gruppo selezionato come `current_group` viene evidenziato). Se l'utente clicca su un nome di gruppo, quel gruppo diventa il corrente (il client invierà eventualmente un join se non ancora membro, oppure semplicemente mostrerà la chat se già membro).

- Un **pannello di destra**, che mostra la sezione di chat del gruppo selezionato. In cima c'è l'intestazione con il nome del gruppo corrente. A seconda dello stato:
  - Se l'utente **non è ancora membro** di quel gruppo (il suo username non è in `joined_groups`), viene mostrato un pulsante "Join Group" che consente di inviare la richiesta di ingresso.
  - Se invece l'utente **fa parte del gruppo**, viene mostrata la cronologia dei messaggi (presi da `self.messages[group]`) e i controlli della chat. I messaggi sono elencati come testi, con eventualmente un prefisso `<mittente>:` per ogni messaggio normale; i messaggi di sistema (mittente "system" o gruppo "system") possono essere formattati in modo speciale (es. corsivo o colore diverso) per distinguerli.
  - In entrambi i casi, se `inviting_user` è true (cioè se l'utente ha richiesto di invitare qualcuno), al posto dei normali controlli di chat compare un piccolo form per l'invito: un campo di testo per inserire l'username da invitare e i bottoni "Confirm Invite" / "Cancel".
  - Se invece non si sta invitando nessuno (`inviting_user == false`), vengono mostrati i bottoni normali: "Invite User" (per aprire il popup di invito) e "Leave Group" (per abbandonare il gruppo corrente). In fondo al pannello c'è sempre l'area di input per il nuovo messaggio (campo di testo "Message" e pulsante "Send") tramite cui l'utente può scrivere e inviare un messaggio al gruppo.

Questo layout a due colonne consente di interagire agevolmente: l'utente può creare o selezionare gruppi nella lista a sinistra e contemporaneamente visualizzare e partecipare alla chat del gruppo selezionato sulla destra. I popup per la creazione di gruppo e l'invito utente compaiono sovrapponendosi parzialmente alla UI quando attivati, grazie ai flag di stato. L'uso di Iced semplifica la gestione di queste condizioni: il **rendering condizionale** di componenti (come mostrare o nascondere il popup) viene realizzato

semplicemente controllando i booleani `creating_group` e `inviting_user` nel modello e costruendo la vista di conseguenza. Questo approccio declarativo mantiene separata la logica (stato e aggiornamenti) dalla presentazione grafica.

- **Update (Aggiornamento dello stato):** la logica di gestione degli eventi nel client GUI risiede nella funzione `update(MessageGUI)`. Iced utilizza il tipo `MessageGUI` (definito dal programmatore) per rappresentare **tutti gli eventi** a cui l'applicazione reagisce. Nel caso di Ruggine, `MessageGUI` è un enum che include varianti per ogni azione utente (es. `LoginInput` per la digitazione nel campo di login, `LoginSubmit` per il click sul pulsante di login, `CreateGroupPopup` per l'apertura del popup, `ConfirmCreateGroup` per la conferma creazione, ecc.) *e* per ogni evento di rete importante (ad esempio `Received(Message)` per un messaggio ricevuto dal server, o `ReceivedRx` quando è pronto il canale di ricezione). La funzione `update` effettua un match su questi eventi e aggiorna lo stato di conseguenza, spesso innescando anche azioni asincrone (come l'invio di messaggi al server). Vediamo i casi principali gestiti:
  - **Login:** quando l'utente preme il pulsante "Login" (che genera MessageGUI::LoginSubmit), la funzione update controlla che il campo `login_input` non sia vuoto. Se c'è un username inserito, lo salva nel campo `self.username` e avvia la connessione al server. In pratica chiama `Command::perform(...)` per eseguire asincronamente la funzione di rete `connect_and_listen("127.0.0.1:8080")`. Questa funzione stabilisce la connessione TCP e restituisce un `UnboundedReceiver<Message>` (canale dei messaggi in arrivo). Inoltre, subito dopo la connessione, il client invia anche il messaggio di `Login` al server tramite `send_message(&Message::Login { username })`. Quando la `connect_and_listen` completa (nel contesto asincrono di Iced ciò avviene in background), il risultato (il ricevitore `rx`) viene incapsulato in un evento `MessageGUI::ReceivedRx(rx)` che verrà gestito da update. Il case `MessageGUI::ReceivedRx(rx)` semplicemente salva il channel `rx` dentro lo stato `self.rx`, così che poi la `subscription` (di cui diremo) possa utilizzarlo. In sintesi, il flusso di login per il client GUI è: acquisire username → connettersi e iniziare ad ascoltare → inviare al server il messaggio di login.

- **Creazione di un gruppo:** quando l'utente clicca "Create Group", il client genera un evento MessageGUI::CreateGroupPopup che semplicemente imposta `creating_group = true` per far comparire nell'interfaccia il popup di inserimento nome. L'utente digita il nome (che finisce in `group_input`) e preme il pulsante di conferma, generando MessageGUI::ConfirmCreateGroup. A questo punto la update verifica che `group_input` non sia vuoto, quindi invia due messaggi al server: prima un `Message::Create { group }` e subito dopo un `Message::Join { username, group }`. In questo modo, il gruppo viene creato e contemporaneamente il client richiede di entrarvi (evitando che resti solo invitato). In parallelo, localmente il nuovo gruppo viene aggiunto all'insieme `self.groups` e impostato come `current_group`, e `creating_group` torna false (chiusura popup). Il risultato è che il client anticipa l'aggiornamento dell'interfaccia per mostrare subito il gruppo creato e la relativa chat, in attesa poi di ricevere gli Ack dal server.
- **Invito di un utente:** similmente, premendo "Invite User" l'app genera MessageGUI::InvitePopup che attiva il popup di invito (`inviting_user = true`). Dopo che l'utente inserisce un nome e preme conferma (MessageGUI::ConfirmInvite), la update legge il campo `invite_input` e, se non vuoto, invia al server un `Message::Invite { group: current_group, user: <invite_input> }`. Non c'è un feedback immediato locale in questo caso, se non la chiusura del popup (inviting_user = false); l'eventuale esito (errori di autorizzazione o l'arrivo di un invito per l'altro utente) si rifletterà tramite messaggi successivi.
- **Join/Leave di un gruppo:** quando nella UI compare il pulsante "Join Group" (visibile se l'utente seleziona un gruppo a cui è invitato ma non ancora entrato) e l'utente lo clicca, viene generato MessageGUI::JoinGroup. La update risponde inviando un `Message::Join { username, group }` al server. Non aggiunge subito il gruppo a joined_groups localmente (aspetta l'Ack). In caso di "Leave Group" (visibile se l'utente è membro), il click genera MessageGUI::LeaveGroup e la update invia un `Message::Leave { group }`. Subito dopo, il client rimuove localmente quel gruppo da `self.joined_groups` e anche da `self.groups` (così sparisce dall'elenco se non si è più invitati), anticipando l'effetto. Anche qui, la conferma definitiva arriverà dal server via Ack.

- **Invio di un messaggio in chat:** quando l'utente digita nel campo di testo del messaggio e preme "Send", l'evento MessageGUI::SendMessage viene inviato alla update. Il client verifica di avere un `current_group` selezionato e che l'utente sia membro di quel gruppo (`joined_groups.contains(current_group)`); se sì, costruisce un `Message::Text { group, from: self.username, content: self.chat_input }` e lo invia al server tramite `send_message`. Poi svuota la propria casella di input `chat_input` (in modo che la UI sia pronta per un nuovo messaggio). Non aggiunge immediatamente il messaggio alla chat locale – questo avverrà quando il server lo rinvierà indietro a tutti i membri, compreso il mittente, e il client lo riceverà tramite il canale di rete.

Ognuna di queste azioni utente in genere comporta due effetti: uno esterno (invio di un messaggio di comando al server) e uno interno (aggiornamento dello stato locale, ad es. selezionare un gruppo corrente, svuotare un campo, chiudere un popup). Grazie al modello architetturale, queste modifiche allo stato causeranno un *rerender* della vista corrispondente (es. comparirà il nuovo gruppo nell'elenco, scomparirà il popup, ecc.), mantenendo l'interfaccia sincronizzata con lo stato applicativo.

- **Ricezione dei messaggi dal server:** oltre agli eventi generati dall'utente, il client GUI deve gestire gli eventi provenienti dalla rete, ossia i messaggi che il server invia (testi di chat degli altri, inviti, conferme, errori). Come anticipato, al momento del login viene impostato `self.rx` come canale di ricezione (`UnboundedReceiver<Message>`) collegato al socket. Iced consente di ascoltare eventi asincroni tramite le **Subscription**. Nel nostro caso, l'applicazione implementa la funzione `subscription()` in modo tale che, se `self.rx` è presente, venga attivata una sottoscrizione che monitora il canale dei messaggi ricevuti dal server e per ognuno genera un evento `MessageGUI::Received(Message)`. In termini semplici, la subscription sposta i messaggi dal thread di rete al thread della GUI. Quando la update riceve un caso `MessageGUI::Received(msg)` (dove `msg` è un `common::Message` proveniente dal server), lo gestisce in base al tipo:

  - Se `msg` è `Message::Text` (un messaggio di chat ricevuto): il client controlla se il campo `group` di tale messaggio corrisponde al gruppo attualmente visualizzato dall'utente (`self.current_group`). Se sì, formatta il messaggio e lo aggiunge al **log dei messaggi** di quel gruppo in `self.messages[group]`. In particolare potrebbe inserire la stringa "`<mittente>: <contenuto>`" nella lista. Se

il messaggio proviene dal “sistema” (es. notifica di join/leave con from = "system" o group = "system"), il client può decidere di mostrarlo stilato in modo diverso o aggiungerlo anch’esso al log come informazione. Se il gruppo del messaggio non esisteva ancora in `self.groups` (ad esempio potrebbe arrivare un Message::Text di sistema per un gruppo che l’utente ha appena lasciato, o simili), il client può aggiungerlo lo stesso alla mappa per completezza. Infine, la vista verrà automaticamente aggiornata per mostrare il nuovo messaggio se il gruppo è visibile.

- Se `msg` è `Message::Invite`: questo è inviato dal server per notificare che **l’utente locale è stato invitato** in un certo gruppo. Il client verifica che `user` nel messaggio corrisponda al proprio username (per sicurezza; nel nostro protocollo il server invia l’invito solo al diretto interessato). Se corrisponde, aggiunge il nome del `group` ricevuto all’insieme `self.groups`. Ciò farà comparire quel gruppo nella lista laterale dell’interfaccia, magari evidenziato come “nuovo” (non implementato esplicitamente, ma il solo fatto che appaia segnala all’utente che ha un invito). L’utente a questo punto potrà decidere se ignorare o cliccare sul gruppo ed eseguire Join.
- Se `msg` è `Message::Ack`: indica che un’operazione invocata in precedenza è andata a buon fine. Il client può usare queste conferme per aggiornare lo stato sapendo che il server ha completato l’azione. Nel progetto Ruggine, ad esempio, quando arriva un Ack in seguito a un Join, il client aggiunge quel gruppo alla propria lista `joined_groups` e, se è il gruppo attualmente selezionato, potrebbe aggiungere un messaggio di sistema locale tipo "*Sei entrato nel gruppo*." per confermare all’utente. Dopo un Ack di Create group, il client sa che il gruppo è effettivamente creato (ma in quel caso lo aveva già aggiunto alla lista in anticipo). Un Ack dopo un Leave può far rimuovere definitivamente il gruppo dall’elenco se non c’è più nemmeno l’invito.
- Se `msg` è `Message::Error`: il client lo elabora mostrando all’utente la notifica di errore. Nel nostro caso, la scelta è stata di rappresentare l’errore come un **messaggio di sistema** visibile nella chat corrente. Ad esempio, se l’utente prova a unirsi a un gruppo senza invito e il server risponde con Error "non sei invitato", il client, trovandosi magari ancora sul gruppo in questione, aggiungerà un messaggio di sistema "*Impossibile unirsi: non*

sei stato invitato..." nella finestra di chat, in modo che l'utente capisca perché la sua azione non ha avuto effetto. Analogamente per altri errori. Questo è uno dei possibili approcci; in alternativa si sarebbero potuti mostrare popup modali o banner di errore.

Va sottolineato come l'utilizzo di canali asincroni e subscription renda la gestione degli eventi di rete **non bloccante**: i messaggi in arrivo dal server vengono processati **nel loop di update** di Iced, evitando di toccare direttamente la UI da un thread separato, il che mantiene il modello di thread safety di Iced e lascia l'interfaccia sempre reattiva. In altre parole, il thread principale della GUI non è mai bloccato in attesa di dati dal socket, ma viene "notificato" quando ci sono nuovi messaggi attraverso il meccanismo delle subscription.

- **Modulo di Rete (client):** gran parte della logica di comunicazione col server nel client GUI è demandata a un modulo separato (`network.rs`) per mantenere pulito il codice UI. Questo modulo fornisce due funzioni fondamentali:
  - `connect_and_listen(addr: &str) → UnboundedReceiver<Message>` – che si occupa di aprire la connessione TCP all'indirizzo del server specificato e predisporre l'ascolto asincrono dei messaggi. Internamente, questa funzione effettua la `TcpStream::connect`, poi esegue `stream.into_split()` per ottenere `reader` e `writer`. Il `writer` viene inserito in una variabile globale statica (`OnceCell`) come `Arc<Mutex<OwnedWriteHalf>>` (chiamata `WRITER`). Questo permette di avere un **accesso globale** al writer condiviso, così che anche da altre parti del codice (es. la funzione `send_message`) sia possibile scrivere sul socket senza doverlo passare in giro come parametro. Si crea poi un **canale mpsc** non limitato (`unbounded_channel`) e si ottiene tx/rx. Viene lanciato un nuovo task Tokio dedicato alla ricezione: in un loop, legge ogni riga dal `reader` (ugualmente con BufReader) e per ciascuna linea ricevuta la deserializza in un `Message` e la invia tramite `tx`. Se il socket si chiude, il loop termina automaticamente. Infine, la funzione `connect_and_listen` ritorna il ricevitore `rx` al chiamante. In questo modo abbiamo incapsulato in un'unica chiamata tutto il setup della connessione di rete e abbiamo ottenuto un **canale asincrono** su cui arriveranno i messaggi dal server.
  - `send_message(msg: &Message)` – che serve per inviare un messaggio applicativo al server. Questa funzione serializza l'enum `Message` passato in input in una stringa JSON (usando Serde) e vi aggiunge un `\n` finale. Dopodiché

accede alla variabile globale `WRITER`: se è inizializzata, ne ottiene il lock (Mutex) e scrive la stringa sul socket con `write_all`, inviandola al server. In caso `WRITER` non sia stato impostato (il che significherebbe che `connect_and_listen` non è stato chiamato, situazione anomala), la funzione stampa un errore su stderr e non fa nulla. Questo metodo semplifica molto il codice dell'interfaccia: qualunque parte del client che voglia inviare un messaggio (login, create, ecc.) può semplicemente costruire un `Message` e chiamare `send_message(&msg)` senza preoccuparsi dei dettagli del socket.

Grazie a queste astrazioni, il **codice GUI rimane pulito e reattivo**: ad esempio, durante il login, la GUI ottiene subito il `rx` dal `connect_and_listen` e registra la subscription, mentre l'invio del Message::Login avviene in background; similmente per tutti gli altri comandi l'update invoca `send_message` in modo fire-and-forget. L'arrivo dei messaggi dal server attraverso il channel `rx` viene gestito in modo **asincrono** e non bloccante rispetto al thread principale Iced. Questo design decoupla completamente la logica di rete (socket, thread I/O) dal loop grafico, migliorando l'**affidabilità** e la facilità di mantenimento del codice.

Client Testuale (CLI)

Come accennato nell'introduzione, Ruggine include anche un **client testuale** (a linea di comando) creato soprattutto per scopi di test e debugging iniziali. Si tratta di un programma console, eseguibile separatamente (crate `client`), che consente di inviare comandi al server e ricevere output testuale, senza interfaccia grafica. Questa sezione descrive brevemente il funzionamento di tale client CLI e come interagisce col server.

Il client testuale utilizza anch'esso Tokio per gestire la comunicazione in modo asincrono, ma l'"interfaccia utente" in questo caso è il terminale (STDIN per input utente e STDOUT per l'output). Quando l'applicazione CLI viene avviata, esegue i seguenti passi:

1. **Richiesta credenziali:** all'avvio, il programma chiede all'utente di inserire il proprio **username** da utilizzare nel sistema. Viene stampata una riga di prompt ("Inserisci il tuo username:") e si legge l'input da stdin (bloccante solo finché l'utente digita, ma nessun altro task viene eseguito nel frattempo). Ottenuto lo username (viene fatto `trim` per rimuovere newline), il client lo conserva in una

variabile. Viene anche predisposta una variabile per il nome del **gruppo corrente** (inizialmente vuota o dummy), che eventualmente verrà impostata quando l'utente si unirà a un gruppo.

2. **Connessione al server:** una volta ottenuto lo username, il client stabilisce la connessione TCP verso l'indirizzo del server (di default `127.0.0.1:8080`) usando `TcpStream::connect().await`. Anche qui viene utilizzato `.into_split()` per ottenere un `reader` e un `writer` separati. Il `reader` verrà avvolto in un BufReader per leggere linee, mentre il `writer` (metà scrittura) verrà usato per inviare dati.
3. **Login iniziale:** subito dopo la connessione, il client CLI invia il messaggio di **login** al server. Costruisce un `Message::Login { username: ... }` con lo username raccolto e lo serializza in JSON tramite `serde_json::to_string`. Aggiunge un `\n` e invia la stringa sul socket usando `writer.write_all().await`. Questo corrisponde al client GUI che preme il tasto Login: registra l'utente sul server. Fatto ciò, il client attende la risposta immediata del server:
   - **Help del server:** il server, come visto, in risposta al login invia un messaggio di testo contenente la lista dei comandi disponibili. Il client CLI legge la prima linea di risposta dal BufReader (bloccante asincrono). Prova a deserializzarla in un `Message` e, se risulta essere un `Message::Text` con un contenuto, lo stampa semplicemente su schermo così come è. In pratica l'utente della console vedrà elencati i comandi possibili (ad esempio "/create <gruppo> ..." etc.). A questo punto il login è completo dal punto di vista dell'utente: è connesso e può interagire.
4. **Task di ricezione asincrona:** il client lancia un task in background per ascoltare continuamente eventuali **messaggi dal server**. Questo task esegue un loop su `buf_reader.read_line(&mut line).await`, simile a quanto fa il server. Per ogni linea ricevuta:
   - Se `read_line` restituisce 0 (EOF), significa che il server ha chiuso la connessione, quindi il task si interrompe (break).
   - Altrimenti, rimuove il newline da `line` e tenta di deserializzare il JSON in un `Message`. Se la deserializzazione ha successo, il programma distingue due casi per stampare un feedback:
     - Se è un `Message::Text` e il campo `from` del messaggio coincide con il nome utente locale (`recv_username`, salvato prima), allora quel messaggio

è di fatto un *eco* di un messaggio che l'utente stesso ha inviato e che il server sta restituendo (questo succede perché il server inoltra i messaggi di chat anche al mittente). In tal caso, il client stampa in console `"Inviato: <messaggio>"`, così che l'utente veda conferma di cosa ha mandato.

  - Negli altri casi (messaggi provenienti da altri utenti, inviti, ack, error), stampa `"Ricevuto: <messaggio>"`, mostrando l'intero enum deserializzato. Questo formato è un po' grezzo (la stampa di Debug di `Message`), ma include tutte le info.

- Se il JSON non era parsabile in un Message (caso raro, potrebbe succedere se il server invia qualcosa di inatteso), il client stampa direttamente la linea raw ricevuta.
- Il loop continua in attesa di altri messaggi. In questo modo, il terminale dell'utente riceverà in tempo reale tutte le notizie: inviti (es. `Invite{group: X, user: Y}`), messaggi di chat di altri (`Text{group: X, from: Bob, content: ...}`), conferme (`Ack`) o errori (`Error{reason: ...}`).

5. **Loop di input utente (invio comandi):** mentre il task sopra ascolta i messaggi in arrivo, il thread principale del client CLI entra in un loop per leggere continuamente l'input da tastiera dell'utente. Viene creato un `BufReader` anche per stdin e si usa `lines()` per ottenere uno Stream di linee di input. Per ogni linea inserita dall'utente (trimmed per togliere newline), il client la interpreta come **comando**:

   - Se la linea comincia con `"/msg "` allora il resto viene interpretato come un comando *invia messaggio*: il testo dopo `"/msg "` dovrebbe contenere il nome di un gruppo e il contenuto del messaggio separati da spazio. Il client esegue uno `split_once(' ')` per dividere la stringa in due parti: la prima parola è considerata `grp` (nome gruppo) e il resto dopo lo spazio `msg_text` (contenuto). Se il formato è valido (entrambi presenti), costruisce un `Message::Text { group: grp, from: username, content: msg_text }` e lo serializza in JSON. Aggiunge newline e lo invia sul socket via `writer.write_all().await`. In più, poiché il server rimanderà indietro lo stesso messaggio, il client *non* stampa subito nulla (opzione: potrebbe stampare "(tu): ..." ma nel nostro caso affida al meccanismo di echo di cui sopra il

compito, che stamperà "Inviato: ..."). Dopodiché torna ad attendere il prossimo input.

- Se la linea inizia con `"/invite "`: analogo al sopra, il client estrae `<grp>` e `<user>` dopo il comando. Se presenti, crea `Message::Invite { group: grp, user: user }`, serializza e invia. In questo caso stampa anche a console un messaggio tipo `"Inviato: Invite{...}"` per feedback (nel codice vediamo un println dopo l'invio).
- Se la linea inizia con `"/join "`: estrae il `<group>` e aggiorna la variabile locale del gruppo corrente (`group = newg`). Crea `Message::Join { username, group: newg }`, serializza, invia sul socket e stampa conferma di invio.
- Se la linea inizia con `"/create "`: estrae il `<group>` da creare. Invia solo `Message::Create { group: newg }` **senza** eseguire join locale immediatamente (il client testuale lascia che sia l'utente poi eventualmente a fare join manualmente, a differenza del GUI che fa join automatico). Stampa un messaggio "Inviato (create): ...".
- Se la linea inizia con `"/leave "` (anche se dal codice fornito non abbiamo visto il blocco, è plausibile implementato analogamente): si estrarrebbe `<group>` e si manderebbe `Message::Leave { group }` e magari si stampa feedback.
- Se la linea inserita non corrisponde a nessuno dei comandi riconosciuti (ad esempio l'utente scrive un testo libero senza prefisso), il client potrebbe ignorarla o stampare un messaggio di errore locale. Nel codice attuale, ogni comando viene gestito da un `if let Some(...) = input.strip_prefix("/comando ")` quindi se nessuno matcha, semplicemente il loop ripete senza fare nulla (dunque input non riconosciuti sono ignorati silenziosamente).

Questo meccanismo a comandi testuali consente all'utente sviluppatore di simulare le azioni dei vari client digitando direttamente in console. Ad esempio, può aprire due terminali, lanciare il client testuale in entrambi, fare login con nomi diversi e usare comandi `/create` e `/invite` per creare un gruppo e invitare l'altro, poi `/join` e scambiarsi `/msg`. Il client mostrerà in tempo reale i "Ricevuto: ..." e "Inviato: ..." corrispondenti, verificando il funzionamento del server.

Il client CLI, pur spartano, è dunque **perfettamente interoperabile** col server come il client GUI, dato che usa lo stesso protocollo JSON e la stessa crate `common` per i messaggi. Serve soprattutto come strumento di debug: essendo a linea di comando, permette di testare rapidamente vari casi, e grazie ai log testuali aiuta a vedere esattamente cosa il server manda/riceve. Dal punto di vista implementativo, mostra anche un esempio di utilizzo di Tokio con standard input/output asincroni e senza la complessità aggiuntiva di un toolkit GUI.

Considerazioni Finali e Possibili Estensioni

Il progetto Ruggine implementa con successo un sistema di chat di gruppo **distribuito** utilizzando strumenti moderni di programmazione di sistema in Rust. Nel corso di questo manuale sono state illustrate nel dettaglio l'architettura client-server adottata, la suddivisione in moduli e componenti, i meccanismi di comunicazione e le strutture dati impiegate, nonché le ragioni dietro certe scelte progettuali. L'insieme copre vari concetti chiave: I/O asincrono su rete, gestione concorrente di dati condivisi, sincronizzazione tra task, interfacce utente reattive e protocolli applicativi personalizzati. Questa base fornisce un ottimo punto di partenza su cui costruire eventuali funzionalità aggiuntive in futuro.

Tra i **miglioramenti futuri** che si potrebbero considerare per evolvere Ruggine verso un utilizzo in scenari reali vi sono ad esempio:

- **Memorizzazione persistente dei dati:** attualmente tutti gli utenti, gruppi e inviti risiedono in memoria volatile e vanno persi se il server termina. Introdurre un database o una persistenza su disco permetterebbe di mantenere lo stato (gruppi creati, storici messaggi, registrazioni utenti) tra riavvii del server.
- **Gestione avanzata delle connessioni:** al momento se un client si disconnette improvvisamente, il server si limita a loggare l'evento e la sua struttura `users` potrebbe conservare riferimenti ormai invalidi. Si potrebbe implementare una pulizia automatica delle strutture dati al momento della disconnessione (rimozione utente da `users`, eventuali inviti non più validi, ecc.). Inoltre, funzioni di **reconnessione** (ad esempio se la connessione cade temporaneamente) e gestione dello stato "offline/online" degli utenti sarebbero utili in un contesto reale.

- **Sicurezza delle comunicazioni:** attualmente le comunicazioni client-server avvengono in chiaro (protocollo TCP non criptato). Per un uso pubblico si dovrebbe prevedere l'uso di TLS/SSL per cifrare i messaggi in transito e proteggere le credenziali e i contenuti delle chat da intercettazioni.
- **Maggiore modularità e scalabilità del server:** con l'aumentare delle funzionalità potrebbe essere opportuno separare il server in più componenti interni, ad esempio distinguendo un modulo per l'**autenticazione/autorizzazione**, uno per la gestione degli **inviti e gruppi**, uno per la trasmissione dei **messaggi** (routing), ecc. Ciò migliorerebbe la manutenibilità del codice. Inoltre, per migliorare la scalabilità si potrebbero investigare strutture dati concorrenti più sofisticate (come usare lock separati per ogni gruppo, o strutture lock-free) o architetture multi-threaded in cui più thread lavoratori accettano connessioni in parallelo. In questo progetto, essendo pensato per scopi dimostrativi e con carichi modesti, si è privilegiata la chiarezza con una singola loop e locking centralizzato, ma in un contesto enterprise si potrebbe adottare un design a shard/partizioni per gestire meglio carichi elevati.

In conclusione, Ruggine costituisce un esempio funzionante di applicazione di chat **multi-utente** basata su Rust, mostrando come utilizzare efficacemente Tokio per la concorrenza, come serializzare semplicemente i messaggi con JSON e come costruire un client grafico reattivo con Iced che interagisce in tempo reale col server. Il manuale ha documentato approfonditamente le sue fondamenta tecniche e concettuali, fornendo una base solida per comprenderne il funzionamento e per eventuali sviluppi successivi in direzione di un sistema ancora più completo e robusto.